
2020-2021 Resit Exam

CPT104 Operating Systems

Exam Answers & Explanations

Generated by OClaw

Question 1 (39 marks)

Q: [12 marks] 1. Compare RAID levels 0, 1, 5, and 6.

RAID 0 (Striping):

- Data split across disks with no redundancy
- Best performance, but NO fault tolerance
- If any disk fails, all data is lost

RAID 1 (Mirroring):

- Exact duplicate of data on 2 disks
- Excellent fault tolerance (can survive 1 disk failure)
- 50% capacity waste

RAID 5 (Block Striping with Distributed Parity):

- Data and parity striped across all disks
- Good read performance, acceptable write performance
- Can survive 1 disk failure
- Only 1 disk worth of capacity overhead

RAID 6 (Dual Distributed Parity):

- Two sets of parity calculations
- Can survive 2 simultaneous disk failures
- Higher write overhead than RAID 5
- 2 disks worth of capacity overhead

[CHN] RAID 0 (条带化)：数据分布在多盘，无冗余，性能最佳但无容错。

RAID 1 (镜像)：数据精确复制到2块盘，极佳容错（可承受1盘故障），50%容量浪费。

RAID 5 (分布式奇偶校验条带化)：数据和校验分布在所有盘上，可承受1盘故障，仅1盘容量开销。

RAID 6 (双重分布式奇偶校验)：两套校验计算，可同时承受2盘故障，写开销高于RAID 5，2盘容量开销。

Q: [6 marks] 2. Explain busy waiting and compare with other waiting mechanisms.

Busy waiting (Spinlock):

- Process continuously checks a condition in a tight loop
- Consumes CPU cycles while waiting
- Wasteful but has zero latency when condition becomes true

Other waiting mechanisms:

(1) Blocking/Sleep: Process is moved to a waiting queue and surrenders CPU. OS wakes the process when the resource is available. No CPU waste, but has context switch overhead.

(2) Interrupt-driven: Hardware notifies the OS via interrupt when an event occurs (e.g., I/O complete). OS then wakes the waiting process. Efficient for I/O operations.

[CHN] 忙等待（自旋锁）：

- 进程在紧密循环中持续检查条件
- 等待期间消耗CPU周期
- 浪费但条件变为真时零延迟

其他等待机制：

(1) 阻塞/休眠：进程移入等待队列并释放CPU。资源可用时OS唤醒。不浪费CPU，但有上下文切换开销。

(2) 中断驱动：硬件通过中断通知OS（如I/O完成），OS再唤醒等待进程。对I/O操作高效。

Q: [6 marks] 3. Why must we restrict memory access? Give three reasons.

(1) Protect the OS kernel from user processes - prevent malicious or buggy user programs from corrupting or reading kernel code/data.

(2) Protect user processes from each other (isolation) - one process should not be able to read or modify another process's memory, ensuring privacy and data integrity.

(3) Prevent illegal access to hardware I/O registers - user processes must not directly access hardware ports or memory-mapped I/O, which could crash the system or cause security violations.

[CHN] (1) 保护OS内核免受用户进程侵害——防止恶意或有缺陷的用户程序损坏或读取内核代码/数据。

(2) 进程间隔离——一个进程不应能读写另一个进程的内存，确保隐私和数据完整性。

(3) 防止非法访问硬件I/O寄存器——用户进程不能直接访问硬件端口或内存映射I/O，否则可能崩溃系统或造成安全违规。

Q: [6 marks] 4. Compare processes and threads.

Process:

- Independent execution unit with its own address space, open files, and resources
- Heavyweight: creation and context switching are expensive
- Processes are isolated from each other
- Inter-process communication (IPC) requires OS mechanisms (pipes, shared memory, etc.)

Thread:

- Lightweight execution unit within a process
- Shares the process's address space, open files, and resources with other threads
- Creation and switching are cheaper than processes
- Communication between threads is simpler (shared memory)
- One thread crash can crash the entire process

[CHN] 进程:

- 独立执行单元, 有自己的地址空间、打开的文件和资源
- 重量级: 创建和上下文切换开销大
- 进程间相互隔离
- 进程间通信需要OS机制 (管道、共享内存等)

线程:

- 进程内的轻量级执行单元
- 与同进程的其他线程共享地址空间、文件和资源
- 创建和切换开销比进程小
- 线程间通信更简单 (共享内存)
- 一个线程崩溃可能导致整个进程崩溃

Q: [4 marks] 5. Explain Role-Based Access Control (RBAC).

Role-Based Access Control (RBAC):

- Permissions are assigned to ROLES rather than individual users
- Users are then assigned to one or more roles
- Simplifies administration: instead of managing permissions for each user, manage permissions per role
- Example: 'Manager' role gets read/write access; 'Intern' role gets read-only access
- When a user changes position, just change their role assignment
- Reduces errors and ensures principle of least privilege

[CHN] 基于角色的访问控制 (RBAC) :

- 权限分配给角色而非个人用户
- 用户被分配一个或多个角色
- 简化管理: 不需要为每个用户管理权限, 而是按角色管理
- 示例: '经理'角色获得读写权限; '实习生'角色获得只读权限
- 用户变更职位时只需更改角色分配
- 减少错误并确保最小权限原则

Q: [5 marks] 6. Explain context switching.

Context switching is the process of saving the state of a currently running process and loading the state of another process so it can run.

Steps involved:

- (1) Save the current process's CPU context: all register values, Program Counter (PC), Stack Pointer (SP)
- (2) Update the current process's PCB (Process Control Block) with its state
- (3) Select the next process to run (scheduling)
- (4) Load the new process's saved state from its PCB: registers, PC, SP
- (5) Update MMU (Memory Management Unit) settings for the new process's address space
- (6) Transfer control to the new process

This is an EXPENSIVE operation due to the overhead of saving and restoring state.

[CHN] 上下文切换是保存当前运行进程状态并加载另一个进程状态使其运行的过程。

步骤:

- (1) 保存当前进程CPU上下文: 所有寄存器值、程序计数器(PC)、栈指针(SP)

- (2) 用当前状态更新进程控制块(PCB)
- (3) 选择下一个要运行的进程 (调度)
- (4) 从新进程PCB加载保存的状态: 寄存器、PC、SP
- (5) 更新MMU设置以匹配新进程的地址空间
- (6) 将控制权转移给新进程

由于保存和恢复状态的开销, 这是一个昂贵的操作。

Question 2 (37 marks)

Q: [6 marks] 1. LRU page replacement: ref=5 0 2 1 0 3 0 2 4 3 0 3 2 1 3 0 1 5, frames=3.

LRU (Least Recently Used) replaces the page that hasn't been used for the longest time.

Step-by-step trace (frames=3):

5: [5,-,-] FAULT | 0: [5,0,-] FAULT | 2: [5,0,2] FAULT
 1: [0,2,1] FAULT | 0: HIT (most recent) | 3: [2,1,3] FAULT
 0: [1,3,0] FAULT | 2: [3,0,2] FAULT | 4: [0,2,4] FAULT
 3: [2,4,3] FAULT | 0: [4,3,0] FAULT | 3: HIT
 2: [3,0,2] FAULT | 1: [0,2,1] FAULT | 3: [2,1,3] FAULT
 0: [1,3,0] FAULT | 1: HIT | 5: [3,0,5] FAULT

Total page faults: 15

[CHN] LRU (最近最少使用) 替换最长时间未使用的页面。

逐步追踪 (3帧) :

5:缺页[5,-,-] | 0:缺页[5,0,-] | 2:缺页[5,0,2]
 1:缺页[0,2,1] | 0:命中 | 3:缺页[2,1,3]
 0:缺页[1,3,0] | 2:缺页[3,0,2] | 4:缺页[0,2,4]
 3:缺页[2,4,3] | 0:缺页[4,3,0] | 3:命中
 2:缺页[3,0,2] | 1:缺页[0,2,1] | 3:缺页[2,1,3]
 0:缺页[1,3,0] | 1:命中 | 5:缺页[3,0,5]

总缺页次数: 15

Q: [9 marks] 2. TLB EAT: TLB=30ns, memory=90ns, hit=70%. What hit rate needed for EAT=130ns?

Current calculation:

TLB hit path: $30 + 90 = 120$ ns

TLB miss path: $30 + 90 + 90 = 210$ ns

Current EAT = $0.7 * 120 + 0.3 * 210 = 84 + 63 = 147$ ns

For target EAT = 130 ns:

$130 = h * 120 + (1 - h) * 210$

$130 = 120h + 210 - 210h$

$130 = 210 - 90h$

$90h = 210 - 130 = 80$

$h = 80 / 90 = 0.889$

Required hit rate = 88.9% (approximately 89%)

[CHN] 当前计算:

TLB命中路径: $30+90=120$ ns

TLB未命中路径: $30+90+90=210\text{ns}$

当前EAT = $0.7*120 + 0.3*210 = 84+63 = 147\text{ns}$

目标EAT=130ns:

$$130 = h*120 + (1-h)*210$$

$$90h = 80$$

$$h = 80/90 = 0.889$$

所需命中率 = 88.9% (约89%)

Q: [7 marks] 3. Memory fragmentation: P1=19900, P2=19990, P3=19888, partition=20000. Can 200 bytes fit?

Each process is placed in a partition of 20000 bytes:

Internal fragmentation:

- P1: $20000 - 19900 = 100$ bytes wasted
- P2: $20000 - 19990 = 10$ bytes wasted
- P3: $20000 - 19888 = 112$ bytes wasted
- Total internal fragmentation = $100 + 10 + 112 = 222$ bytes

Can 200 bytes fit?

NO. The 222 bytes of waste are TRAPPED inside the already-allocated partitions. They are not contiguous and cannot be combined. There is no single free partition of 200 bytes available.

This demonstrates the key problem with internal fragmentation: wasted space cannot be reclaimed without deallocating entire partitions.

[CHN] 每个进程放入20000字节的分区:

内部碎片:

- P1: $20000-19900=100$ 字节浪费
- P2: $20000-19990=10$ 字节浪费
- P3: $20000-19888=112$ 字节浪费
- 总内部碎片 = $100+10+112 = 222$ 字节

200字节能放进去吗?

不能。222字节的浪费被锁在已分配的分区内部，不连续，无法合并。没有200字节的可用空闲分区。

这展示了内部碎片的关键问题：浪费的空间无法在不释放整个分区的情况下回收。

Q: [10 marks] 4. SJF non-preemptive scheduling: P1(arr=0,burst=9), P2(arr=1,burst=5), P3(arr=2,burst=3), P4(arr=3,burst=4).

Non-preemptive SJF (Shortest Job First):

Gantt Chart:

|P1(0-9)|P3(9-12)|P4(12-16)|P2(16-21)|

At t=0: Only P1 available. P1 runs (burst=9, finishes at t=9)

At t=9: P2(burst=5), P3(burst=3), P4(burst=4) all waiting. Shortest = P3. P3 runs (9-12)

At t=12: P2(5), P4(4). Shortest = P4. P4 runs (12-16)

At t=16: P2 runs (16-21)

Turnaround time (finish - arrival):

P1: $9-0=9$, P2: $21-1=20$, P3: $12-2=10$, P4: $16-3=13$

Average turnaround = $(9+20+10+13)/4 = 52/4 = 13$

Waiting time (turnaround - burst):

P1: $9-9=0$, P2: $20-5=15$, P3: $10-3=7$, P4: $13-4=9$

Average waiting = $(0+15+7+9)/4 = 31/4 = 7.75$

[CHN] 非抢占式SJF (最短作业优先) :

甘特图: |P1(0-9)|P3(9-12)|P4(12-16)|P2(16-21)|

t=0: 仅P1可用, 运行(9个单位)

t=9: P2(5),P3(3),P4(4)等待, 最短=P3, 运行(9-12)

t=12: P2(5),P4(4), 最短=P4, 运行(12-16)

t=16: P2运行(16-21)

周转时间: P1:9, P2:20, P3:10, P4:13, 平均=13

等待时间: P1:0, P2:15, P3:7, P4:9, 平均=7.75

Q: [5 marks] 5. LOOK disk scheduling: requests=6,10,12,54,97,73,128,15,44,110,34,45, head=23, dir=decreasing.

LOOK services requests in current direction, then reverses (doesn't go to disk end).

Head at 23, direction = decreasing (toward lower tracks).

Down: 23 -> 15 -> 12 -> 10 -> 6

Movement: $8+3+2+4 = 17$

Reverse (increasing): 6 -> 34 -> 44 -> 45 -> 54 -> 73 -> 97 -> 110 -> 128

Movement: $28+10+1+9+19+24+13+18 = 122$

Total head movement = $17 + 122 = 139$ cylinders

[CHN] LOOK按当前方向服务请求, 然后反转 (不走到磁盘尽头)。

磁头在23, 方向=递减 (向低磁道)。

向下: 23->15->12->10->6, 移动=17

反转 (递增): 6->34->44->45->54->73->97->110->128, 移动=122

总磁头移动量 = $17+122 = 139$ 柱面

Question 3 (12 marks) - Banker's Algorithm

Q: [12 marks] Banker's Algorithm: Determine system safety state and find a safe sequence.

Steps to solve:

1. Calculate Need matrix: Need = Max - Allocation for each process
2. Check if system is in a safe state using the safety algorithm:
 - Start with Available resources
 - Find a process whose Need $\leq$ Available
 - Pretend to allocate, add its Allocation to Available
 - Repeat until all processes finish (safe) or no process can proceed (unsafe)
3. If safe, the order of process completion is the safe sequence
4. If unsafe, the system cannot guarantee deadlock avoidance

Note: Specific numbers depend on the original exam's Max and Allocation matrices. Apply the algorithm systematically to determine the result.

[CHN] 解题步骤:

1. 计算Need矩阵: $\text{Need} = \text{Max} - \text{Allocation}$
2. 使用安全算法检查系统是否安全:
 - 从Available资源开始
 - 找一个 $\text{Need} \leq \text{Available}$ 的进程
 - 假设分配, 将其Allocation加到Available
 - 重复直到所有进程完成 (安全) 或无法继续 (不安全)
3. 若安全, 进程完成顺序即为安全序列
4. 若不安全, 系统无法保证避免死锁

注意: 具体数值取决于原题的Max和Allocation矩阵。系统地应用算法确定结果。

Question 4 (12 marks) - Semaphores & Synchronization

Q: [12 marks] Two processes with semaphores S and R:

P1: wait(S), wait(R), x++, signal(S), signal(R)

P2: wait(R), wait(S), x--, signal(S), signal(R)

(a) Can deadlock occur? (b) Can indefinite postponement occur?

(a) Deadlock: YES, absolutely possible.

Scenario:

- P1 executes wait(S) - acquires S
- P2 executes wait(R) - acquires R
- P1 tries wait(R) - BLOCKED (P2 holds R)
- P2 tries wait(S) - BLOCKED (P1 holds S)
- Circular wait: P1 holds S, waits R; P2 holds R, waits S
- This is a CLASSIC deadlock from lock ordering violation

(b) Indefinite postponement (starvation): YES, possible.

If the scheduler always favors one process (e.g., always runs P1), the other process (P2) may never get CPU time to complete its operations. This is a scheduling fairness issue, not a correctness issue.

[CHN] (a) 死锁: 是的, 完全可能。

场景:

- P1执行wait(S)——获取S
- P2执行wait(R)——获取R
- P1尝试wait(R)——阻塞 (P2持有R)
- P2尝试wait(S)——阻塞 (P1持有S)
- 循环等待: P1持有S等待R, P2持有R等待S
- 这是经典的锁顺序违规导致的死锁

(b) 无限期推迟 (饥饿): 是的, 可能。

如果调度器总是偏向一个进程 (如总运行P1), 另一个进程(P2)可能永远得不到CPU时间。这是调度公平性问题, 而非正确性问题。